

End Report for CI/CD Pipeline Project

COMP.SE.140

Sayem Rahman

Instructions for the Teaching Assistant

This document provides an overview of the CI/CD pipeline implementation, testing, and deployment process. It also includes reflections on the challenges encountered during the development and testing phases.

Implemented Optional Features

- Testing of individual services
- Static analysis step in the pipeline using Pylint
- Monitoring and logging for troubleshooting
- Deployment to an external server

Instructions for Examiner to Test the System

Links of the repositories:

Gitlab:

<https://compse140.devops-gitlab.rd.tuni.fi/SayemRahman/Docker-Exercise/>

Github:

<https://github.com/SayemRa/Docker-Exercise/tree/project>

1. Clone the repository:

<https://github.com/SayemRa/Docker-Exercise.git>

<https://compse140.devops-gitlab.rd.tuni.fi/SayemRahman/Docker-Exercise.git>

2. Build and start the application:

```
docker-compose build --no-cache  
docker-compose up -d
```

3. Run API tests using curl:

```
curl localhost:8197/state -X PUT -d "RUNNING" -H "Content-Type: text/plain"  
curl localhost:8197/state -X GET
```

4. Run unit tests:

```
python -m unittest discover -s tests
```

Development Platform

- **OS:** Windows 10
- **CPU:** Ryzen 5 5700x
- **Docker Version:** 24.0.7

- **Docker-Compose Version:** 1.29.2
- **Python Version:** 3.10

Description of the CI/CD Pipeline

Version Management

- The development was carried out in the `project` branch to ensure that previous branches remained unaffected.

Building Tools

- Docker and Docker-Compose were used to containerize and build the services.

Testing

- **Unit Tests:** Implemented for Service1, Service2, and integration.
- **Static Analysis:** Pylint was used to enforce code quality standards.
- **Integration Tests:** Ensured proper communication between services.

Packing

- The application was packaged into Docker images and managed using Docker-Compose.

Deployment

- The pipeline automates deployment by starting containers when new code is pushed.
- Deployment was tested locally and on an external server.

Operating and Monitoring

- A monitoring endpoint (`/monitor`) logs service uptime and request count.
- The `/run-log` endpoint tracks state transitions.

Example Runs of the Pipeline

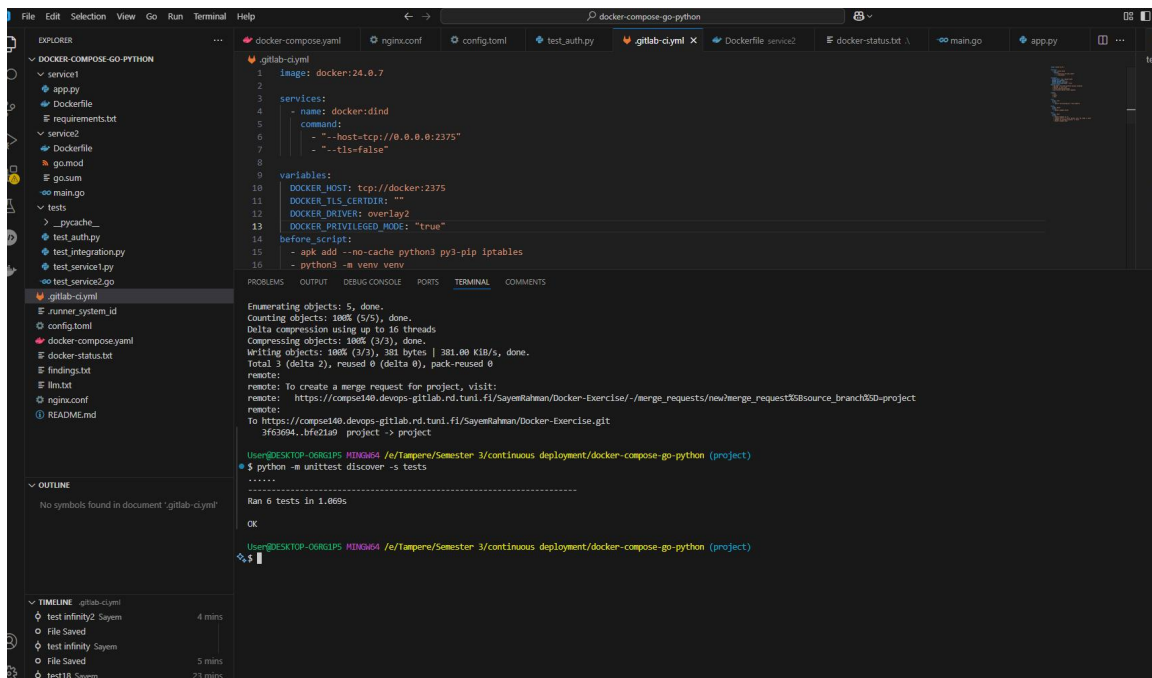
Successful Run

```
```sh
pylint service1/app.py --fail-under=5
Passed

docker-compose build
Build successful

docker-compose up -d
Services started successfully

python -m unittest discover -s tests
All tests passed
```
```



Failing Run (Due to GitLab Constraints)

- The pipeline encountered resource constraints in GitLab CI/CD, leading to test failures.
- The same tests passed successfully in the local development environment.

Reflections

Main Learnings

- Gained hands-on experience with CI/CD pipeline implementation.
- Understood the importance of resource allocation in cloud CI/CD environments.
- Learned best practices for test-driven development and containerization.

Worst Difficulties

1. GitLab CI/CD Pipeline Failures:

- Tests failed in GitLab due to resource constraints, while they passed locally.
- This could be mitigated by using a self-hosted runner with increased resources.

Even after continuous trying, I failed to complete the gitlab procedures.

2. Network Dependencies:

- Ensuring inter-service communication in Docker was challenging.

Improvement Suggestions

- Implement load testing to identify potential bottlenecks.
- Use a dedicated cloud environment with more resources for CI/CD execution.
- Improve logging and debugging capabilities for pipeline troubleshooting.

Effort Estimation

- **Development & Implementation:** 30 hours
- **Testing & Debugging:** 40 hours
- **Documentation:** 2 hours
- **Total:** 72 hours